

1. Setup and Run instructions:

Note: Testing Locally on Windows as i have a PC and no Linux/Mac access

Setup : Downloads and installations

- .NET 8 SDK from - <https://dotnet.microsoft.com/en-us/download/dotnet/thank-you/sdk-8.0.424-windows-x64-installer>
- Node.js 18+ from - <https://nodejs.org/en/download>
- VS code from - https://code.visualstudio.com/download?_exp_download=fb315fc982

Run :

- clone the tic-tac-toe project to say for e.g. C:\dev\tic-tac-toe
- Open PowerShell
- **Run the backend:**
 - launch powershell
 - cd C:\dev\tic-tac-toe\backend\TicTacToe.Api

`dotnet restore`

`dotnet run`

- here if you get 'AddSwaggerGen' error try add the package by running command 'dotnet add package Swashbuckle.AspNetCore'

- after a while it should be listening on port 5000

Now listening on: http://localhost:5000

- **Start the frontend:**
- Open a new second PowerShell window to run front end:

`npm install`

Then after npm install run the below command

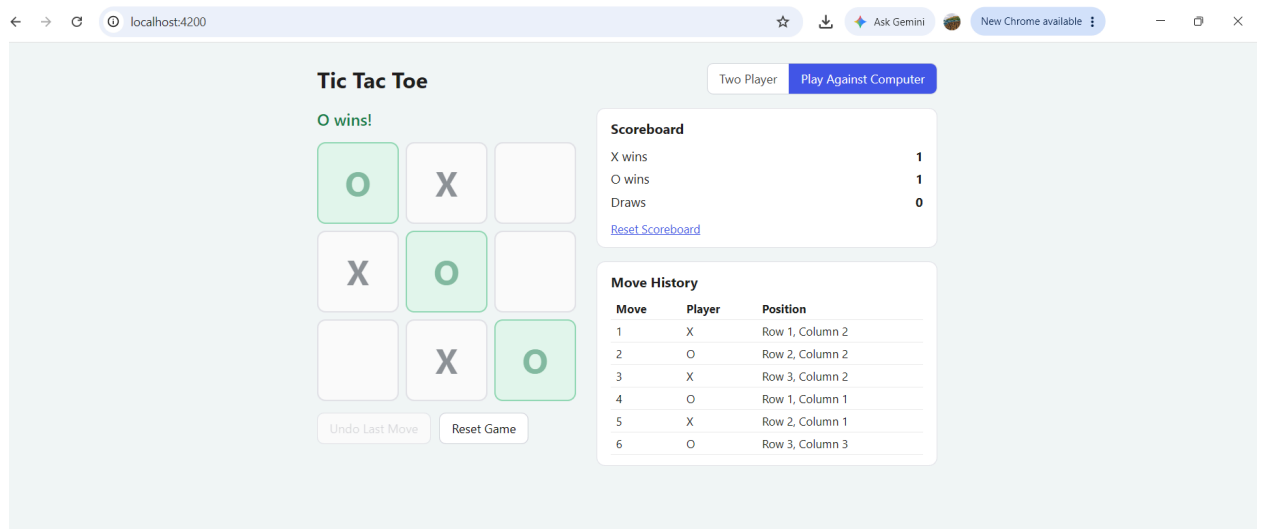
`npm start`

This command may give prompt like Would you like to share pseudonymous usage data about this project with the Angular Team at Google under Google's Privacy Policy at <https://policies.google.com/privacy>. For more details and how to change this setting, see <https://angular.io/analytics>. Choose y or N

This runs ng serve with the proxy config pointed at your backend. Wait for:

after ✓ Compiled successfully message, launch browser and go to `http://localhost:4200` in your browser and the game will run or the appropriate URL in the powershell prompt.

Game looks like this



2. Test Instructions

Backend (xUnit):

powershell

`cd C:\dev\tic-tac-toe\backend`

`dotnet test`

Covers: valid/invalid moves, turn switching, row/column/diagonal wins, draw detection, reset, undo in both modes (incl. edge cases — no moves, after completion), scoreboard update-once, computer move-selection priority.

Frontend (Karma/Jasmine):

powershell

```
cd C:\dev\tic-tac-toe\frontend
```

```
npm test
```

Covers: board rendering and click interaction (emits index, ignores disabled/filled cells, highlights winning cells), and `AppComponent`'s API integration (game creation, move submission, error-message surfacing) via `HttpClientTestingModule`. Runs in watch mode by default — for a single non-interactive pass:

```
powershell
```

```
npm test -- --watch=false --browsers=ChromeHeadless
```

Manual/end-to-end check: in one PowerShell window run `dotnet run` from `C:\dev\tic-tac-toe\backend\TicTacToe.Api`; in a second, run `npm start` from `C:\dev\tic-tac-toe\frontend`. Open `http://localhost:4200` and play a full game in both modes to confirm win/draw/undo/reset/scoreboard behave as expected in the browser.

3. Prompt Summary / AI Workflow Notes

- **Tool used:** Claude
- **Approach:**

Here is the prompt:

Build a browser-based Tic Tac Toe application with an Angular frontend and a .NET backend running locally.

Build a complete, working solution as below

1. Backend: .NET 8 Web API implementing all functional requirements — game board, player turns, win detection (rows/columns/diagonals), draw detection, reset game, move history, undo (mode-aware: single move in Two Player mode, move-pair in Vs Computer mode), scoreboard (X wins/O wins/draws, update-once per game, separate reset), and a computer opponent following this exact priority: win if possible → block opponent's win → take center → take a corner → take any cell. The backend must own all game state, validate every move server-side, and expose the REST endpoints from the "Suggested API Scope" table (or an equivalent contract, clearly documented).

2. Frontend: Angular + TypeScript SPA that calls the backend for every action and renders exactly what the backend returns — board, current player, mode, winner/draw message, highlighted winning cells, move history, scoreboard, and Reset Game / Undo Last Move / Reset Scoreboard controls. The frontend should not implement any game rules itself.

3. Resolve Clarification 2 (Scoreboard and Undo) by picking Option A or B and documenting the choice.

4. Tests: backend unit tests (xUnit) covering at minimum — valid move, invalid move, turn switching, row/column/diagonal win, draw, reset, undo in both modes, scoreboard update, computer move selection, and move-after-completion. Add frontend tests covering component rendering and API integration.

5. Deliver a full project structure (backend + frontend + tests) as actual files, not just descriptions, along with a README covering: project overview, tech stack, features implemented, how to run the backend locally, how to run the frontend locally, API endpoint summary, how to run tests, AI tools and prompt summary, design decisions, clarifications and assumptions, known limitations, and future improvements.

- **What AI generated:** the codebase — backend models/services/controllers/tests, frontend models/service/components/tests, and documentation.
- **What was changed/verified manually:**
 - Fixed a missing `Swashbuckle.AspNetCore` NuGet package reference that caused a build failure (AI wrote code calling Swagger APIs but didn't declare the package dependency) — caught when I ran `dotnet run` locally on Windows and fixed with `dotnet add package Swashbuckle.AspNetCore`.
 - Ran `dotnet test` and `npm test` locally on my Windows PC to confirm the generated logic and specs actually pass, since the AI authored this without a live .NET/Angular toolchain to compile against.
 - Hit and resolved a PowerShell execution-policy prompt during `npm install (Set-ExecutionPolicy -Scope CurrentUser RemoteSigned)`.
 - Reviewed `GameService.MakeMove/Undo` and `ComputerPlayerService.ChooseMove` line-by-line to be able to explain the validation order, undo-by-mode logic, and computer priority unprompted.
- **Design decisions I own:** chose Option A (disable Undo after game completion) per the assignment's Clarification 2; kept Reset Game on the same game ID rather than minting a new one.

4. API Documentation / Endpoint Summary

Base URL: `http://localhost:5000/api` (Swagger UI at `http://localhost:5000/swagger` in Development)

Method	Endpoint	Purpose	Request body	Success response
--------	----------	---------	--------------	------------------

POST	/games	Create a new game	{ "mode": "TwoPlayer" "VsComputer" }	GameStateResponse
GET	/games/{id}	Get current game state	—	GameStateResponse
POST	/games/{id}/moves	Submit a move	{ "player": "X" "O", "cellIndex": 0-8 } (or row+column instead of cellIndex)	GameStateResponse
POST	/games/{id}/undo	Undo last move (mode-aware)	—	GameStateResponse
POST	/games/{id}/reset	Reset current game (scoreboard kept)	—	GameStateResponse
GET	/scoreboard	Get session scoreboard	—	{ xWins, oWins, draws }
POST	/scoreboard/reset	Reset scoreboard to zero	—	{ xWins, oWins, draws }

GameStateResponse shape:

```

json
{
  "id": "a1b2c3",
  "board": ["X", null, "O", null, "X", null, null, null, null],
  "currentPlayer": "O",
  "mode": "TwoPlayer",
  "status": "InProgress",
  "winner": null,
  "winningCells": [],
  "moveHistory": [{ "moveNumber": 1, "player": "X", "cellIndex": 0, "row": 0, "column": 0 }],
  "canUndo": true,
  "scoreboard": { "xWins": 0, "oWins": 0, "draws": 0 }
}

```

Errors: `400 Bad Request` with `{ "message": "..."}` for invalid moves (occupied cell, out-of-range cell, wrong player's turn, move after completion) or invalid undo (no moves / game completed). `404 Not Found` for an unknown game id.

You can verify this whole table live: open `http://localhost:5000/swagger` in a browser while the backend is running, and every endpoint above shows up with its request/response schema.

5. Known Assumptions and Limitations

Assumptions:

- Undo is **disabled once a game is completed** (Clarification 2, Option A) — the scoreboard is final for a finished game.
- Reset Game reuses the same game ID and keeps the current mode; `POST /games` is the way to start a genuinely new session.
- A move request may supply either `cellIndex` or `row+column` — the backend normalizes both.
- In Vs Computer mode, the computer's reply is applied synchronously within the same `POST /moves` call — the frontend never polls for it.

Limitations:

- **No persistence** — in-memory only; restarting the backend (closing the PowerShell window / stopping `dotnet run`) clears all games and the scoreboard (acceptable per the assignment's storage option).
- **No auth/multi-user isolation** — the scoreboard is one shared, process-wide counter (single local session).
- **No unbeatable AI** — computer opponent uses a 1-ply win/block heuristic, not minimax; it can lose in some lines.
- **Windows-specific setup notes:** requires a fresh PowerShell window after installing the .NET SDK for `dotnet` to be on PATH; `npm install` may require `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` if script execution is restricted; Windows Firewall may prompt to allow `dotnet.exe` network access on first run.